CASE STUDY

Turning Messy HR Data Into a Simple, Useful Dashboard

Role: Data / Business Analyst · **Tools:** Python, Streamlit, Plotly, Pandas · **Scope:** 500+ employee records across 4 cities

I was given raw HR data with no easy way to use it. I checked the data for mistakes, fixed the ones I found, and built a simple dashboard from scratch that helps leaders see, in plain terms, where staff are leaving and why — and why it's happening.

Situation

A company with offices in four cities in Kenya — Nairobi, Mombasa, Kisumu, and Nakuru — had information about all of its staff: who worked there, what they were paid, how long they'd stayed, and who had left. But nobody could actually use this information to answer simple questions, like: *Which department is losing the most people? Are staff choosing to leave, or are they being let go? Is pay fair between men and women, and between the four cities?* The information existed, but it was just sitting there. Nobody had turned it into anything useful.

It got worse when I looked closer. Some of the numbers in the data didn't add up. Two columns that were supposed to show the same thing — whether someone still worked there or not — sometimes disagreed with each other. And it wasn't clear if the pay numbers were monthly or yearly. If these mistakes weren't fixed first, any report built on top of this data could give the wrong answer, and nobody would notice.

Task

My job was to take this raw data, find and fix what was wrong with it, and turn it into an easy-to-use dashboard that could answer the company's real questions — something that could also be shared with anyone, not just people who could log into the live app.

Action

Checked the data for mistakes before trusting it

Found that two columns meant to say the same thing didn't always agree, figured out why (they were being filled in separately instead of one coming from the other), and fixed it so they can't disagree anymore. Also checked whether pay numbers were monthly or yearly, and corrected them so they're clear and consistent.

Built the dashboard around real questions, not just around the columns in the data

Made five simple pages: one page that shows everything at a glance, and four pages that each answer one clear question — like "Who's leaving, and why?" or "Is pay fair across our four cities?" Every chart comes with a short, plain note explaining what it shows and why it matters, instead of just a caption.

Let people click on a chart to see the real details behind it

Added a feature where clicking on a bar in a chart shows the actual staff records behind that number. When it first didn't work, I found the exact reason — the code was reading the wrong part of the chart — and tested the fix to make sure it actually worked before calling it done.

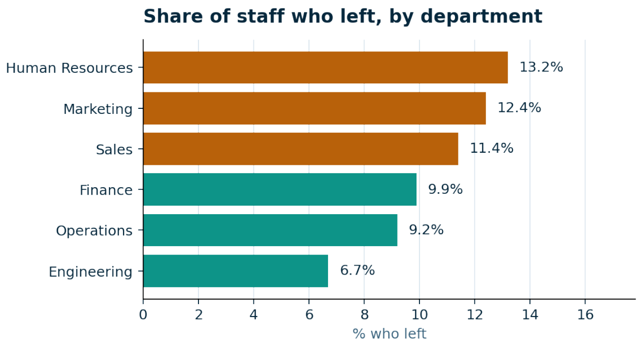
Kept private information private

Staff names are hidden by default everywhere they appear. Only someone with a password can reveal them, and once they enter it, it stays unlocked for the rest of their visit instead of asking again on every page.

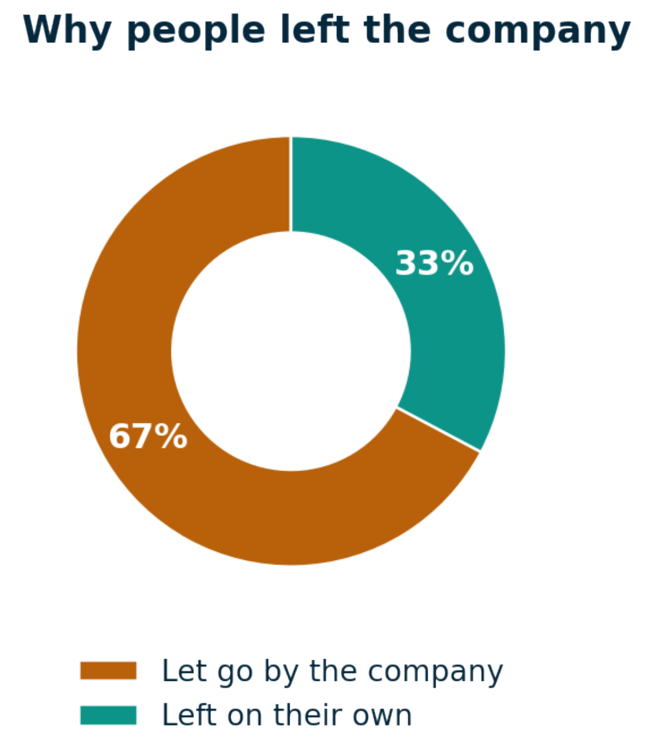
Made it easy to share

Added buttons to download the whole dashboard as a webpage or as an image, so someone can share the findings by email or in a slide without needing to open the live app. Along the way, I also fixed a technical problem that was stopping the image download from working.

Two examples of what the finished dashboard shows:



One department loses almost twice as many staff as the best-performing one.



Most people who left didn't choose to — they were let go.

Result

Found the real story. Most people who left didn't quit — they were let go. That means this is mostly a hiring and performance problem, not a "people don't want to stay" problem.

Found exactly where to look first. One department loses almost twice as many staff as the best one, giving leaders a clear place to start instead of guessing.

Fixed a hidden data problem for good. The two columns that used to disagree can no longer do so, so every number on the dashboard can be trusted.

Made answers fast to find. Anyone can see the big picture on one screen, then click once to dig deeper into the details they care about.

Made it shareable. The whole dashboard can be downloaded as a file and shared with anyone, even if they can't open the live app.

Tested fixes properly. Instead of assuming a fix worked, I checked it directly — for example, by simulating the exact click that was failing before — before calling anything finished.

Live dashboard: **View it here**

This project was built on made-up (sample) data, as a way to show how I work through a data problem from start to finish — not as a claim about results from a real company.